"Validate early. Handle gracefully. Respond consistently."

MODULE 29

Validation and Global Exception Handling

Learn how to validate input data and implement global exception handling in Spring Boot applications to build robust, secure, and user-friendly APIs.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 29 OVERVIEW

Learn how to validate input data and implement global exception handling in Spring Boot applications to build robust, secure, and user-friendly APIs.

Validate early, handle gracefully, respond consistently. This module covers Bean Validation annotations, controller-level @Valid, and centralized @ControllerAdvice exception handling, then unifies error contracts across the Northstar CRM API.

DURATION & LAB

3 Hours Theory

Bean Validation (JSR 380) annotations, the validation lifecycle, and @RestControllerAdvice exception handling.

90 Minutes Lab

Northstar CRM Error Contracts: unify validation and exception handling into one stable response shape.

Scenario

No API error path may return framework-default HTML stack traces or ad-hoc Map bodies to React.

MODULE ARC

Understand Validation

Bean Validation catches invalid data early, before it reaches business logic.

Master Global Handling

@ControllerAdvice and @ExceptionHandler centralize errors into one stable ErrorResponse.

Build the Lab

Annotate CRM DTOs, enable @Valid, and implement GlobalExceptionHandler with correlation IDs preserved.

KEY TAKEAWAY Total time: 3 hours theory + 90 minutes hands-on lab. Effective validation and global exception handling ensure data integrity, system security, and a great developer and user experience.

WHY VALIDATION MATTERS

Validation is the first line of defense in any application. It ensures that only correct, safe, and meaningful data enters your system.

Goal: accept good data, reject bad data early, and provide clear feedback -- this leads to secure, reliable, and user-friendly applications.

Without Validation	With Validation
Invalid data accepted -- incorrect or incomplete data enters the system.	Good data accepted -- only valid, complete data enters the system.
Data quality issues -- corrupted records, inaccurate business data.	Better data quality -- consistent, accurate, reliable data.
Security vulnerabilities -- malicious input causes SQL injection, XSS, etc.	Stronger security -- blocks malicious or harmful input at the boundary.
Poor user experience -- users see errors or confusing messages later.	Better user experience -- clear, helpful messages and smooth interactions.
Higher maintenance cost -- fixing bad data, bugs, and exceptions costs time and money.	Lower cost & higher efficiency -- less rework, fewer errors, better productivity.

THE FOUR-WORD SUMMARY

Validate Early **Save Time** **Reduce Risk** **Deliver Quality**

KEY TAKEAWAY Validation helps you build secure, reliable, and user-friendly APIs by ensuring that the data your application processes is correct from the start.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to implement robust validation and global exception handling in a Spring Boot application.

Understand Validation Fundamentals — explain the importance of validation and the validation lifecycle in a Spring Boot application.

Use Bean Validation Annotations — apply commonly used annotations such as @NotNull, @NotBlank, @Email, and @Size.

Apply Validation in Controllers — integrate validation in REST controllers to ensure data integrity at the API boundary.

Handle Validation Errors — return meaningful and user-friendly validation error messages to API consumers.

Implement Request Validation — validate incoming request data using annotations and handle validation errors effectively.

Implement Global Exception Handling — create centralized exception handling using @ControllerAdvice and @ExceptionHandler.

Design Standard Error Responses — build consistent and user-friendly error response models with proper error codes and messages.

Follow Best Practices — apply best practices for validation, logging, security, and exception handling in enterprise applications.

KEY TAKEAWAY You will be able to implement robust validation and global exception handling to build secure, reliable, and user-friendly Spring Boot applications.

INPUT VALIDATION OVERVIEW

Input validation is the process of verifying that incoming data is complete, correct, and in the expected format before it is processed.

WHY VALIDATE INPUT?

Prevents Invalid Data — stops incomplete, incorrect, or malformed data from entering the system.

Enhances Security — protects against threats like SQL injection, XSS, command injection, and more.

Ensures Data Quality — maintains consistency and accuracy of stored and processed data.

Improves User Experience — provides clear and helpful feedback about invalid inputs.

Reduces System Errors — minimizes runtime exceptions and system failures caused by bad data.

VALIDATION IN THE REQUEST PROCESSING FLOW

VALIDATION CAN HAPPEN AT MULTIPLE LEVELS

- **Client Side**
- **API/Controller**
- **Service Layer**
- **Database Layer**

Good validation is the foundation of great applications: accurate data, a secure application, reliable processing, and a better user experience.

KEY TAKEAWAY Validate early, validate often! Always validate data at the boundaries of your application to build secure, reliable, and user-friendly systems.

Validation Overview

Ensure that input data is correct, complete, and safe before processing.

Goal

Prevent invalid data from entering the system, reduce errors, improve user experience, and enhance application security.

What is Validation?

Validation is the process of checking input data against defined rules and constraints to ensure it meets expected criteria.

Where Validation Happens

1. Input Source

Data comes from clients (web forms, REST APIs, mobile apps, etc.)

2. Validation Layer

Input is validated against business rules and constraints using validation framework (e.g., Bean Validation).

3. Processing

If valid, data proceeds to business logic and processing layers.

4. Outcome

Valid data leads to successful execution and response. Invalid data triggers errors and prevents processing.

5. Feedback

Users receive clear, consistent feedback about validation errors to take corrective action.

Key Benefits

Data Integrity
Ensures only accurate and complete data enters the system.

Better User Experience
Provides immediate and meaningful feedback to users.

Enhanced Security
Protects against invalid, malicious, or unexpected input.

Reduced Errors & Costs
Catches issues early, reducing defects and support overhead.

Efficiency
Prevents unnecessary processing of invalid data.

Remember: Validate early, validate consistently, and always provide clear feedback.

VALIDATION LIFECYCLE

Validation happens at multiple points in the application lifecycle to ensure only correct and safe data is accepted and processed.

1. **Data Entry** — User input
2. **Input Reception** — App receives it
3. **Validation Execution** — Rules applied
4. **Validation Result** — Errors collected
5. **Decision** — Proceed or stop
6. **Processing** — If valid
7. **Response** — Success or errors

TYPES OF VALIDATION

Type	Where
Client-Side	Quick checks in the UI for instant feedback.
Server-Side	Primary validation in controllers/services.
Database	Constraints enforced at the database level.

BENEFITS OF A STRONG VALIDATION LIFECYCLE

- Improved Data Quality
- Enhanced Security
- Reduced Errors & Rework
- Better UX
- Higher Reliability

KEY TAKEAWAY Validate early, validate often. Catch issues as early as possible to save time and cost and improve user experience.

Validation Lifecycle

A systematic process to ensure data quality and integrity at every step.

Purpose

Validate input data early and consistently to prevent invalid data from propagating through the system.

Outcome

High quality data, predictable behavior, better user experience, and stronger application security.

1 Input Received

Application receives input from client (UI, API request, message, file upload, etc.).

2 Pre-Validation

Basic checks such as required fields, format presence, type checks, and data normalization.

3 Validation Execution

Apply validation rules (Bean Validation constraints, custom rules, business rules).

4 Validation Result

Collect and evaluate validation results, including all violations (not just the first one).

5 Error Handling

Handle validation failures gracefully. Determine response format and error details.

6 Respond to Client

Return meaningful, user-friendly error messages with field details and guidance.

7 Logging & Monitoring

Log validation failures for audit, monitoring, and continuous improvement.

Key Principles

Validate Early
Catch issues as early as possible in the request lifecycle.

Validate Consistently
Apply the same rules across all entry points and layers.

Validate Completely
Return all validation errors at once, not just the first.

Communicate Clearly
Provide clear, actionable error messages that users understand.

Improve Continuously
Monitor, analyze and refine validation rules over time.

Remember: A strong validation lifecycle prevents bad data, reduces errors, improves security, and builds user trust.

Receive → Validate → Respond → Monitor

VALIDATION BEST PRACTICES

Following validation best practices helps ensure your application is secure, reliable, and provides a great user experience.

#	Practice	What It Means
1	Validate Early	Validate as early as possible in the request lifecycle; catch invalid data before it enters business logic.
2	Use Built-in Validation	Leverage Bean Validation annotations (@NotNull, @Size, @Email, etc.) instead of hand-rolled checks.
3	Keep Rules Consistent	Apply consistent rules across all layers; avoid duplicate or conflicting validations.
4	Provide Clear Messages	Return user-friendly, actionable error messages; avoid exposing internal details.
5	Validate for Security	Protect against malicious input (XSS, SQL injection, command injection); never trust client input.
6	Validate the Right Things	Validate only what is necessary; don't over-validate or hurt performance.
7	Centralize Validation	Use @Valid and @ControllerAdvice to centralize handling and keep one source of truth for errors.
8	Monitor and Improve	Log validation failures (without sensitive data); review and improve rules based on feedback.

DOs	DON'Ts
Use @Valid on request DTOs. Group related validations. Keep messages consistent. Test rules with unit/integration tests.	Don't trust client-side validation alone. Don't return stack traces. Don't skip validation for performance. Don't expose sensitive info in errors.

KEY TAKEAWAY Good validation is not just about catching errors -- it's about building secure, reliable, and user-friendly applications that keep data integrity and system trust.

BEAN VALIDATION FRAMEWORK

Bean Validation is a standard specification (JSR 380 / Jakarta Bean Validation) for validating Java objects using annotations.

1 **Annotations**
Define rules on DTOs

2 **Hibernate Validator**
Reference implementation

3 **Validation**
Applies rules, reports violations

WHAT IS BEAN VALIDATION?

Standard Specification — JSR 380 (Jakarta Bean Validation) defines a standard for object validation.

Annotation-Based — declares constraints on fields, methods, or classes.

Provider Implementation — Hibernate Validator is the reference implementation used in Spring Boot.

Automatic & Consistent — integrates with Spring MVC/Boot to validate automatically.

KEY COMPONENTS

Constraints — predefined annotations like @NotNull, @Email, @Size, @Min.

Constraint Validators — logic that checks if a value satisfies the constraint.

Validation Engine — processes constraint metadata and performs validation.

Constraint Violations — hold details of validation failures (field, message, invalid value).

HOW IT WORKS

1. Define Constraints — add validation annotations to model/DTO classes.

2. Trigger Validation — the framework validates the object (e.g. on an API request).

3. Evaluate Constraints — each constraint validator checks the value against the rules.

4. Return Violations — if any rule fails, violations are collected and returned.

BENEFITS

- Reduces boilerplate code
- Consistent validation everywhere
- Easy Spring integration
- Clear, user-friendly errors

KEY TAKEAWAY Bean Validation provides a powerful, standard, annotation-driven way to validate data, ensuring clean, consistent, and reliable applications.

Bean Validation

Standardized, declarative validation of Java objects using annotations.

What is Bean Validation?

Bean Validation is a standard (Jakarta Bean Validation / Hibernate Validator) that allows you to declare validation rules on Java beans using annotations.

How Bean Validation Works

1. **Annotate** — Add constraint annotations to bean fields.
2. **Validate** — Validator validates the bean (manually or automatically).
3. **Check Rules** — Each constraint is checked against the field value.
4. **Collect Errors** — All violations are collected into a ConstraintViolation list.
5. **Result** — If no violations, bean is valid and processing continues.

Benefits

- Reduces boilerplate validation code
- Consistent and declarative rules
- Automatic integration with frameworks (Spring, JAX-RS, etc.)
- Improves reliability and user experience

Common Annotations

@NotNull

Field value must not be null.

Example:

```
@NotNull
private String name;
```

@NotBlank

For strings, value must not be null or empty or blank.

Example:

```
@NotBlank
private String email;
```

@Email

Field value must be a well-formed email address.

Example:

```
@Email
private String email;
```

@Size

Size of field (string, collection, array) must be within specified range.

Example:

```
@Size(min = 2, max = 50)
private String username;
```

@Min / @Max

Numeric value must be greater than or equal / less than or equal to the limit.

Example:

```
@Min(18) @Max(100)
private int age;
```

@Pattern

String value must match the given regular expression.

Example:

```
@Pattern(regexp = "[A-Z]{3}-\d{3}")
private String code;
```

Example: Validated Bean

```
import jakarta.validation.constraints.*;

public class Customer {

    @NotNull
    private Long id;

    @NotBlank
    private String name;

    @Email
    private String email;

    @Size(min = 2, max = 50)
    private String username;

    @Min(18) @Max(100)
    private int age;
}
```

Where Bean Validation is Used

- **Spring MVC** — Validate @RequestBody, @RequestParam, and @PathVariable
- **REST APIs** — Ensure request payloads are valid before business logic execution
- **Service Layer** — Validate domain objects before persistence or processing
- **UI Forms** — Validate user input on server-side before returning response

Key Points

- Bean Validation separates validation rules from business logic.
- Annotations make validation declarative and easy to maintain.
- Can be used at field, method parameter, or class level.
- Integrated with frameworks for automatic triggering.

BEAN VALIDATION ANNOTATIONS (1 OF 2)

@NotNull and @NotBlank: the two most common constraints for requiring a value to be present.

@NOTNULL -- PREVENTS NULL VALUES

Ensures a field, method parameter, or return value cannot be null. Applies to any reference type (String, Object, List). Does NOT apply to primitives (int, long).

Customer.java

```
@NotNull(message = "Name must not be null")
private String name;

@NotNull(message = "Email must not be null")
private String email;
```

On failure: a ConstraintViolationException (or MethodArgumentNotValidException on @RequestBody) is thrown, handled by the global exception handler with a 400 Bad Request.

@NOTBLANK -- PREVENTS EMPTY/BLANK STRINGS

Ensures a String is not null AND contains at least one non-whitespace character. Applies only to String types.

User.java

```
@NotBlank(message = "Username must not be blank")
private String username;

@NotBlank(message = "Email must not be blank")
private String email;
```

Input	@NotBlank Result
null	Invalid (fails)
"" (empty string)	Invalid (fails)
" " (spaces only)	Invalid (fails)
"John"	Valid

KEY TAKEAWAY Use @NotNull for any required reference type; use @NotBlank for required text so empty or whitespace-only strings are rejected too.

BEAN VALIDATION ANNOTATIONS (2 OF 2)

@Email and @Size: constraints for format correctness and length/count boundaries.

@EMAIL -- VALIDATES EMAIL FORMAT

Ensures a string is a well-formed email address (RFC 5322). String types only -- combine with @NotBlank for required, valid emails.

CustomerRequest -- combined usage

```
@NotBlank(message = "Email is required")
@Email(message = "Please provide a valid email")
private String email;
```

Input Value	@Email Result
user@example.com	Valid
user@sub.domain.com	Valid
userexample.com	Invalid (missing @)
user@.com / user@domain	Invalid (bad domain)

@SIZE -- VALIDATES LENGTH / ELEMENT COUNT

Validates a String, Collection, Map, or array's size against min/max boundaries (attributes: min, max, message).

User.java

```
@Size(min = 3, max = 30, message = "Username must be between 3 and 30
characters")
private String username;
@Size(min = 1, max = 5, message = "At least 1 and at most 5 roles allowed")
private List<String> roles;
```

Input (length)	@Size(min=3,max=10) Result
"Aman" (4)	Valid
"" (0)	Invalid (too small)
"This is a very long text" (24)	Invalid (too large)

KEY TAKEAWAY @Email validates format; @Size enforces length/count boundaries. Combine both with @NotBlank so fields are present, well-formed, and within range.

TRY IT YOURSELF — EXERCISE 1: DTO CONSTRAINT PLAN

Pre-lab Exercise 1 -- plan Bean Validation constraints for CustomerRequest's four core fields before writing any code.

STEP	WHAT TO DO
Goal	Plan Bean Validation constraints for CustomerRequest's four core fields before writing any code.
Field List	In notes/dto-constraints.md, write a constraint for each of: name, email, customerId, status.
Reference Check	Match your list to the reference: name -> @NotBlank; email -> @Email + @NotBlank; customerId -> @NotBlank + CUS-#### pattern; status -> @NotNull + allowed values.
Starter Dependency	Note that Lab 29 adds spring-boot-starter-validation to pom.xml -- the dependency that activates annotation-based validation.
Boundary	Do not implement the full CustomerRequest class yet -- this is a planning exercise, not a coding one.
Deliverable	notes/dto-constraints.md with all four fields constrained and the validation starter dependency named.

Reference -- target annotations (Lab 29 CustomerRequest)

```
@NotBlank(message = "Name is required")
private String name;
@NotBlank(message = "Email is required") @Email(message = "Please provide a valid email")
private String email;
@NotBlank @Pattern(regexp = "CUS-\\d{4}", message = "Must match CUS-####")
private String customerId;
@NotNull(message = "Status is required")
private CustomerStatus status;
```

TRY IT NOW Complete Exercise 1 before continuing -- see exercises/exercise-01-dto-constraints.md.

REQUEST VALIDATION WORKFLOW

In a Spring Boot application, incoming requests are automatically validated before the controller method body executes.

KEY POINTS

- Validation happens automatically -- no manual checks in the controller.
- All constraint violations are collected and reported together.
- Invalid requests never reach your business logic.
- Proper error responses improve API usability and client experience.

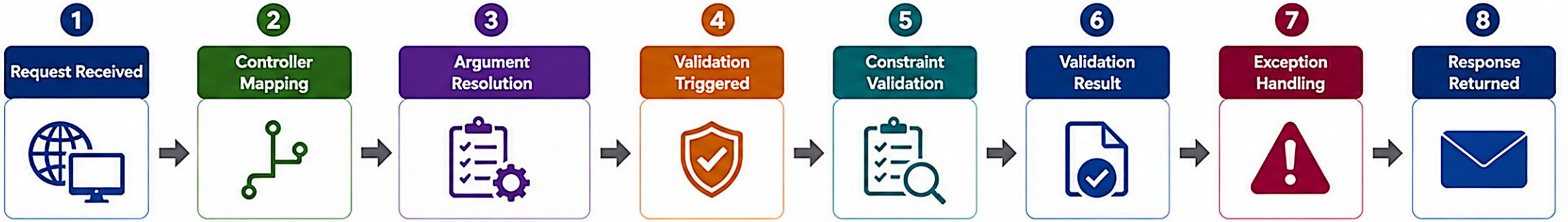
CustomerController.java

```
@PostMapping
public ResponseEntity<CustomerResponse> create(
    @Valid @RequestBody CustomerRequest request) {
  // @Valid triggers validation before this line runs
  return service.create(request);
}
```

KEY TAKEAWAY The request validation workflow ensures bad data is caught early, protecting your application and providing clear feedback to API clients.

Request Validation Workflow

How incoming requests are validated in a Spring Boot application.



Where Validation Happens

@RequestBody with @Valid / @Validated

Validates the request body mapped to a Java object.

```
public ResponseEntity<?> create(
  @Valid @RequestBody
  CreateUserRequest request) { ... }
```

@PathVariable

Validate path variables using @Pattern, @Min, @Max, etc.

```
@GetMapping("/users/{id}")
public User getUser(
  @PathVariable @Min(1) Long id) { ... }
```

@RequestParam / @RequestHeader

Validate query params and headers.

```
public List<User> search(
  @RequestParam @Size(min = 1) String q) { ... }
```

@ModelAttribute

Validate form data bound to a model object.

```
public String submit(
  @Valid @ModelAttribute UserForm form) { ... }
```

Validation Triggers

- @Valid or @Validated on controller method arguments
- Bean Validation annotations on DTO / Model fields
- Method parameter constraints (@Min, @Max, @Pattern, etc.)
- Custom validations (ConstraintValidator)

Common Validation Errors

- MethodArgumentNotValidException
  (Triggered for @RequestBody / @ModelAttribute violations)
- ConstraintViolationException
  (Triggered for @PathVariable, @RequestParam, etc. violations)

Flow Summary

Best Practices

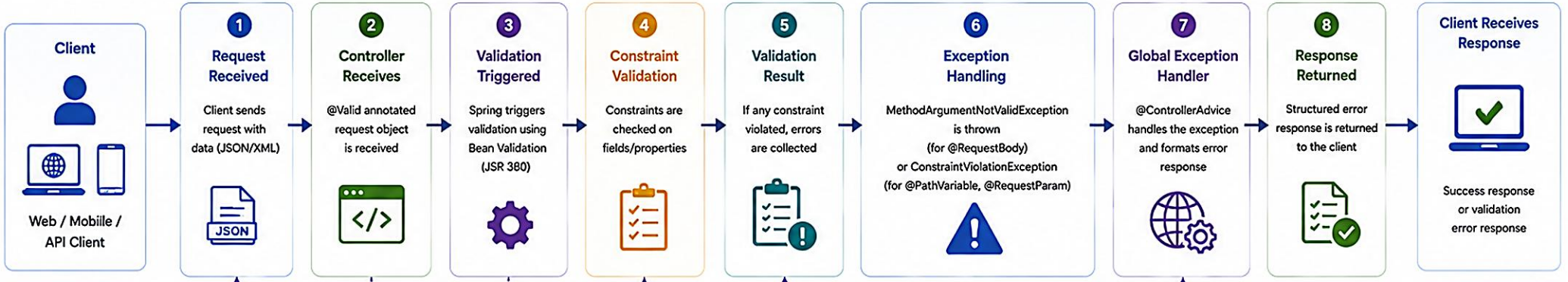
- Always validate at the boundary (API layer).
- Use meaningful messages for constraints.
- Return user-friendly errors with field specific details.
- Use groups for different validation scenarios.
- Log validation failures for audit and monitoring.
- Keep error responses consistent across the application.

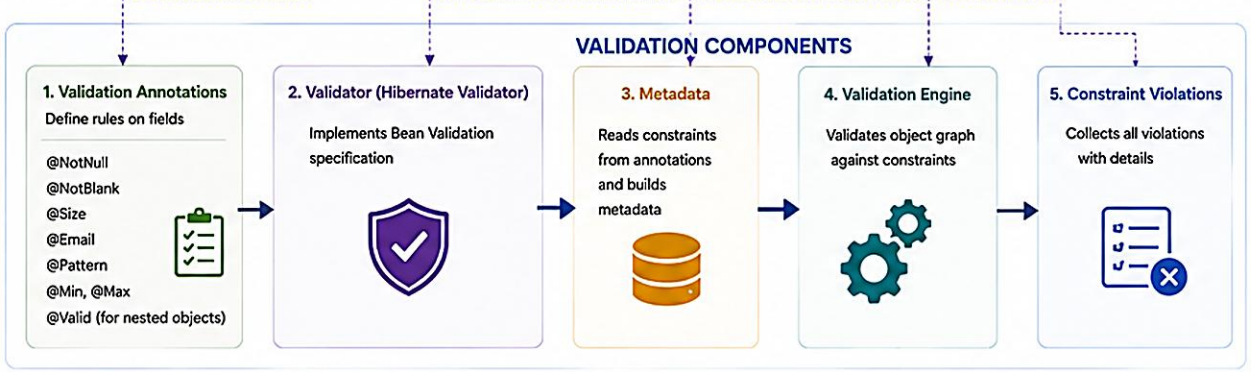
Outcome

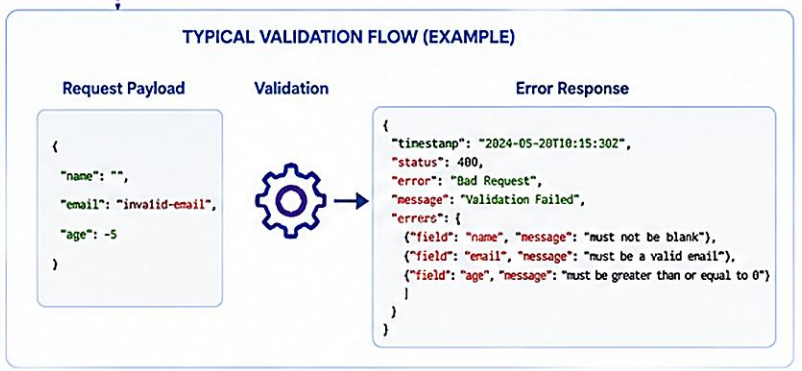
Only valid and safe data enters your application, ensuring data integrity, better UX, and stronger security.

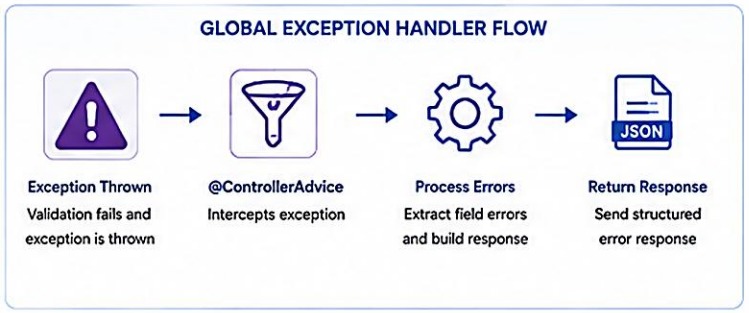
Validation Processing Workflow

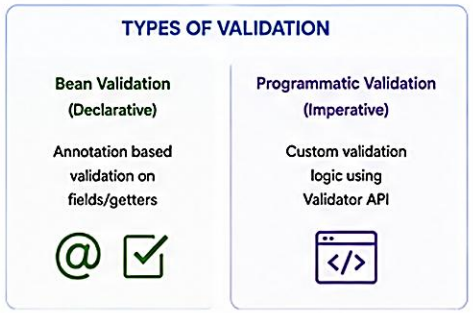
How validation requests are processed in a Spring Boot application

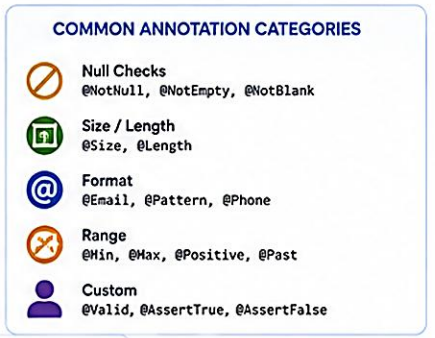












HANDLING VALIDATION ERRORS

When validation fails, Spring throws a ConstraintViolationException or MethodArgumentNotValidException. Handle these to return clear, consistent responses.

COMMON VALIDATION EXCEPTIONS

MethodArgumentNotValidException — thrown when validation fails for @RequestBody, @ModelAttribute, or @RequestPart objects.

ConstraintViolationException — thrown when validation fails for @RequestParam, @PathVariable, or method parameters annotated with @Validated.

GlobalExceptionHandler.java

```
@RestControllerAdvice
public class GlobalExceptionHandler {
  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<ErrorResponse> handleValidation(
      MethodArgumentNotValidException ex) {
    var errors = ex.getBindingResult().getFieldErrors();
    return ResponseEntity.badRequest().body(
        ErrorResponse.of("VALIDATION_FAILED", errors));
  }
}
```

EXAMPLE ERROR RESPONSE

```
{
  "timestamp": "2026-07-14T12:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_FAILED",
  "message": "Validation failed",
  "errors": [
    { "field": "email", "message": "must be a valid email" }
  ]
}
```

Centralized w/ @RestControllerAdvice **Consistent Structure**

Field-Level Messages **No Internal Details**

KEY TAKEAWAY Handle validation errors centrally to provide clear, consistent, and user-friendly feedback. This improves API reliability and client experience.

Handling Validation Errors

Capture, process, and return meaningful validation errors to help clients correct their requests.

Validation Error Handling Flow

1. Validation Fails
Bean Validation detects constraint violations on request data.

2. Constraint Violations
All violations are collected in a BindingResult object (MethodArgumentNotValidException is thrown).

3. Exception Caught
Global exception handler (@ControllerAdvice) catches the exception.

4. Error Processing
Extract field errors, messages, error codes, and other details from the exception.

5. Build Error Response
Create a consistent, user-friendly error response structure.

6. Return to Client
Send structured error response with HTTP status (usually 400).

Common Sources of Validation Errors

Source	Description
Request Body (@RequestBody)	Invalid or missing fields in JSON/XML payload mapped to a Java object.
Path Variables (@PathVariable)	Invalid format or missing values in path variables.
Query Parameters (@RequestParam)	Missing required parameters or invalid values.
Request Headers (@RequestHeader)	Missing or invalid header values.
Model Attributes (@ModelAttribute)	Validation errors in form data bound to model objects.

Exception Flow

MethodArgumentNotValidException (or ConstraintViolationException)

↓

Handled by @ExceptionHandler in @ControllerAdvice

↓

Extract errors from BindingResult (Field errors | Object errors | Messages)

↓

Map to standard error response model

↓

Return ResponseEntity<ErrorResponse> with HTTP 400 Bad Request

Example Error Response (HTTP 400)

```
{
  "timestamp": "2025-05-15T10:30:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/customers",
  "errors": [
    {
      "field": "name",
      "rejectedValue": "",
      "message": "Name must not be blank",
      "code": "NotBlank"
    },
    {
      "field": "email",
      "rejectedValue": "invalid-email",
      "message": "Email must be a valid email address",
      "code": "Email"
    }
  ]
}
```

Key Elements

- **timestamp** — When the error occurred
- **status** — HTTP status code
- **message** — Summary of the error
- **path** — API endpoint that failed
- **errors** — Detailed list of field/object errors

Best Practices for Handling Validation Errors

- **Be Consistent** — Use a standard error response model across all APIs.
- **Provide Clear Messages** — Return user-friendly messages to help clients fix issues.
- **Include Error Codes** — Use machine-readable codes for programmatic handling.
- **Return All Errors** — Return all validation errors in one response, not just the first.
- **Avoid Exposing Internals** — Do not expose stack traces or internal implementation details.
- **Log for Diagnostics** — Log full error details on the server for debugging and monitoring.

Remember

Great error handling improves developer experience, reduces support requests, and increases API reliability.

USER-FRIENDLY ERROR MESSAGES

Good error messages help users understand what went wrong and how to fix it -- clear, specific, and actionable, without exposing internal details.

Scenario	Bad Message (Confusing)	Good Message (Helpful)
Missing username	"Validation failed for field: username"	"Username is required."
Invalid email	"Invalid email format"	"Please enter a valid email address."
Username too short	"Size constraint violated"	"Username must be between 3 and 30 characters."
Weak password	"Password not valid"	"Password must be at least 8 characters and include a number and special character."
Date in past	"Invalid date"	"Date of birth must be in the past."

PRINCIPLES OF USER-FRIENDLY MESSAGES

Be Clear **Be Specific** **Be Actionable** **Be Consistent** **Be Safe**

KEY TAKEAWAY User-friendly error messages improve user experience, reduce support tickets, and build trust in your API.

EXCEPTION HANDLING REVIEW

Exception handling helps your application manage errors gracefully, maintain normal flow, and provide meaningful feedback when things go wrong.

TYPES OF EXCEPTIONS

Checked — checked at compile time; must be caught or declared. Examples: IOException, SQLException.

Unchecked (Runtime) — not checked at compile time, usually programming errors. Examples: NullPointerException, IllegalArgumentException.

EXCEPTION HIERARCHY (SIMPLIFIED)

- Throwable
  - Error -- serious problems the JVM can't handle.
  - Exception
    - Checked Exceptions (compile-time checked).
    - Runtime Exceptions (unchecked).

TRY-CATCH-FINALLY

```
try {
  // risky code here
} catch (SpecificEx e) {
  // handle specific first
} catch (Exception e) {
  // fallback for others
} finally {
  // always runs (cleanup)
}
```

WHEN TO THROW EXCEPTIONS

Invalid Input/State | **Business Rule Violations** | **External System Failures** | **Unexpected Internal Errors**

KEY TAKEAWAY Exception handling is not just about catching errors -- it's about building resilient applications that recover, respond, and keep users informed.

GLOBAL EXCEPTION HANDLING OVERVIEW

Global exception handling provides a centralized mechanism to handle exceptions across the entire application and return consistent responses.

1. **Client**
2. **Controller**
3. **Service / Repository**
4. **Exception Occurs**
5. **Global Handler**
6. **Error Response**

WHY GLOBAL EXCEPTION HANDLING?

Centralized

Handle all exceptions in a single location.

Consistent Responses

Return a uniform error structure across all APIs.

Cleaner Code

Controllers stay focused on business logic.

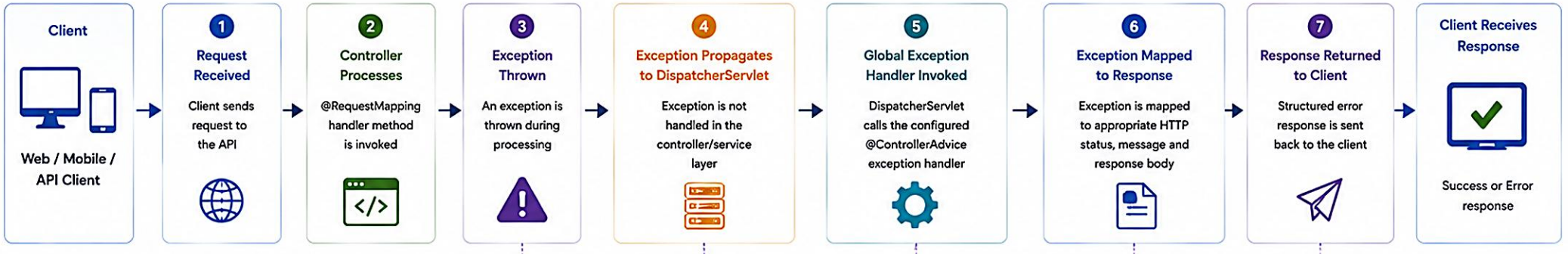
COMMON EXCEPTIONS HANDLED

Exception	When It Occurs
MethodArgumentNotValidException	@RequestBody / @ModelAttribute failed.
ConstraintViolationException	@RequestParam / @PathVariable failed.
ResourceNotFoundException	Requested resource not found.
Custom Business Exceptions	Domain rule violations.
Exception (generic)	Unexpected server errors.

KEY TAKEAWAY Global exception handling centralizes error management, ensures consistent and user-friendly responses, and improves reliability and user experience.

Global Exception Handling Flow

Centralized Exception Handling in Spring Boot Applications



How Global Exception Handling Works (@ControllerAdvice)

@ControllerAdvice (Global Exception Handler)
A global component that contains @ExceptionHandler methods for different exception types.

@ExceptionHandler Methods
Methods are mapped to handle specific exception classes. (e.g., RuntimeException, ValidationException)

Build Error Response
Create a consistent error response using custom structure. (status, message, timestamp, path, details, errors)

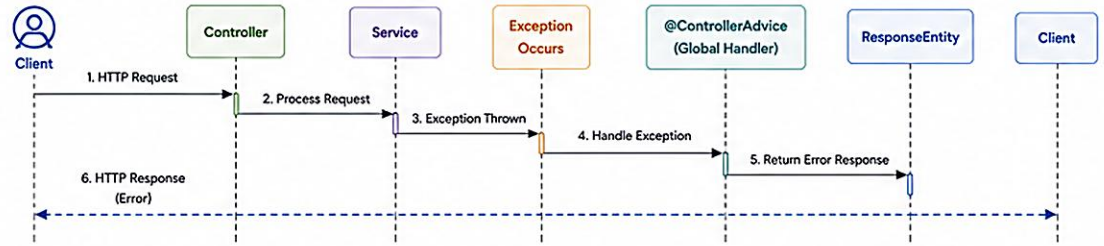
Return ResponseEntity
Return ResponseEntity with appropriate HTTP status and response body.

DispatcherServlet Sends Response
Response is written to HttpServletResponse and sent back to the client.

Example Exception Mapping

Exception Type	HTTP Status	When Used
ResourceNotFoundException	404 NOT FOUND	Resource not found
ValidationException	400 BAD REQUEST	Invalid input data
UnauthorizedException	401 UNAUTHORIZED	Authentication required
AccessDeniedException	403 FORRIDDEN	Insufficient permissions
BusinessException	422 UNPROCESSABLE ENTITY	Business rule violation
Exception (Generic)	500 INTERNAL SERVER ERROR	Unhandled server error

Typical Flow (Example)



Standard Error Response Structure (Example)

```
{
 "timestamp": "2024-05-20T10:15:30Z",
 "status": 400,
 "error": "Bad Request",
 "message": "Validation failed",
 "path": "/api/users",
 "details": {
   { "field": "email", "message": "must be a valid email" },
   { "field": "age", "message": "must be >= 18" }
 }
}
```

Benefits

- Centralized error handling
- Consistent error responses
- Separation of concerns
- Easier maintenance
- Better debugging & logging
- Improved API reliability

Best Practices

- Handle specific exceptions before generic ones.
- Return meaningful messages without exposing internal details.
- Use a consistent error response structure across the application.
- Log exceptions properly for monitoring and debugging.

Common Annotations in Exception Handling

@ControllerAdvice
Marks global exception handler class

@ExceptionHandler
Handles specific exception types

@ResponseStatus
Defines HTTP status for exceptions

@RestControllerAdvice
Combination of @ControllerAdvice and @ResponseBody

Sample Global Exception Handler Skeleton

```
@RestControllerAdvice
public class GlobalExceptionHandler {
   @ExceptionHandler(ResourceNotFoundException.class)
   public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
      ErrorResponse response = new ErrorResponse(
         HttpStatus.NOT_FOUND.value(), "Resource Not Found", ex.getMessage(),
         LocalDateTime.now(), request.getRequestURI());
      return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
   }
}
```

Global Exception Handling

Centralized, consistent, and user-friendly error handling across the entire application.

What is Global Exception Handling?

Global exception handling provides a centralized mechanism to handle exceptions thrown from any controller or layer and return a consistent error response.

How Global Exception Handling Works

1. **Request Processing** — Client sends a request to the application (REST API).
2. **Exception Thrown** — An exception occurs in controller, service, repository, or any layer.
3. **Exception Propagation** — Controller ↑ Service ↑ Repository. Exception propagates up the call stack.
4. **Caught by @ControllerAdvice** — Global exception handler (@ControllerAdvice) intercepts the exception.
5. **Handle & Build Response** — @ExceptionHandler method handles the exception and builds an error response.
6. **Response Returned** — Structured error response is returned to the client with appropriate HTTP status code.

Example: Global Exception Handling in Spring Boot

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidationException(
            MethodArgumentNotValidException ex) {
        List<FieldError> fieldErrors = ex.getBindingResult().getFieldErrors();
        List<ErrorDetail> errors = fieldErrors.stream()
            .map(err -> new ErrorDetail(err.getField(), err.getDefaultMessage()))
            .collect(Collectors.toList());

        ErrorResponse response = new ErrorResponse(
            "VALIDATION_FAILED",
            "Validation failed for one or more fields",
            errors,
            LocalDateTime.now()
        );

        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        ErrorResponse response = new ErrorResponse(
            "NOT_FOUND", ex.getMessage(), List.of(), LocalDateTime.now());
        return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
    }

    // Additional @ExceptionHandler methods for other exceptions...
}
```

Key Annotations

- **@RestControllerAdvice:** Marks the class as a global exception handler for REST controllers.
- **@ExceptionHandler:** Maps specific exception types to handler methods.

Standard Error Response Model

```json
{
  "timestamp": "2025-05-15T10:30:00Z",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_FAILED",
  "message": "Validation failed for one or more fields",
  "path": "/api/customers",
  "errors": [
    {
      "field": "name",
      "rejectedValue": "",
      "message": "Name must not be blank"
    },
    {
      "field": "email",
      "rejectedValue": "invalid-email",
      "message": "Email must be a valid email address"
    }
  ]
}
```

- **timestamp** — When the error occurred
- **status** — HTTP status code
- **error, code, message** — High-level error information
- **path** — API endpoint that failed
- **errors** — Detailed field or object error information

Features

- **Centralized Handling** — Handle exceptions from any layer in one place.
- **Consistent Responses** — Ensure uniform error structure across all APIs.
- **Improved UX** — Return clear, user-friendly messages.
- **Easy Maintenance** — Add, update, and manage exception handling easily.
- **Better Security** — Avoid exposing internal details or stack traces to clients.

Best Practices

- **Be Specific** — Handle specific exceptions instead of generic ones.
- **Use Standard Model** — Return consistent error response structure for all APIs.
- **User-Friendly Messages** — Provide clear messages that help users fix their requests.
- **Avoid Exposing Internals** — Do not expose stack traces, SQL errors, or internal implementation details.
- **Log the Exceptions** — Log full details for diagnostics and support.
- **Test Exception Scenarios** — Test all exception scenarios to ensure proper handling.

Remember

Good global exception handling leads to robust applications, happy users, and easier support and debugging.

@CONTROLLERADVICE ANNOTATION

@ControllerAdvice is a specialization of @Component that lets you handle exceptions and customize responses globally across all controllers.

KEY FEATURES

Global Scope — handles exceptions from all controllers in a single place.

Reusable Logic — avoids code duplication across controllers.

Consistent Responses — ensures a uniform error structure across the API.

Flexible Scoping — can be applied to specific packages or controller groups.

GlobalExceptionHandler.java

```
@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(IllegalArgumentException.class)
  public ResponseEntity<ErrorResponse> handleIllegalArgument(
      IllegalArgumentException ex) {
    ErrorResponse response = new ErrorResponse(
        "INVALID_ARGUMENT", ex.getMessage());
    return ResponseEntity.badRequest().body(response);
  }
}
```

KEY TAKEAWAY @ControllerAdvice helps you handle exceptions in one place, ensuring consistent, maintainable, and user-friendly error handling across your entire application.

@CONTROLLERADVICE

SPRING BOOT

Centralized Exception Handling for All Controllers

@ControllerAdvice (or **@RestControllerAdvice**) allows you to handle exceptions globally across all controllers in a consistent and maintainable way.

WHAT IS @CONTROLLERADVICE?

@ControllerAdvice is a specialized component that enables global exception handling, data binding, model attributes, and custom response logic across all @Controller classes.

HOW @CONTROLLERADVICE WORKS

1. CLIENT REQUEST

Client sends an HTTP request (REST API)

2. CONTROLLER

Request is handled by @RestController endpoint

3. SERVICE LAYER

Business logic is executed in the service layer

4. EXCEPTION

If an exception occurs (e.g., validation, not found, business rule), it is thrown

5. @CONTROLLERADVICE

GLOBAL EXCEPTION HANDLER
Handles exception from all controllers

- Maps exception to specific @ExceptionHandler method
- Builds a standardized error response body
- Sets appropriate HTTP status code
- Returns response to the client

6. HTTP RESPONSE

Standardized JSON error response with HTTP status

EXAMPLE: CONTROLLER

```
@RestController
@RequestMapping("/api/customers")
public class CustomerController {

    @GetMapping("/{id}")
    public ResponseEntity<Customer> getCustomer(
          @PathVariable Long id) {
      Customer customer = customerService.findById(id)
      .orElseThrow(() -> new CustomerNotFoundException(id));
      return ResponseEntity.ok(customer);
   }
}
```

EXAMPLE: CONTROLLERADVICE

```
@RestControllerAdvice
public class GlobalExceptionHandler {

   @ExceptionHandler(CustomerNotFoundException.class)
   public ResponseEntity<ErrorResponse> handleNotFound(
        CustomerNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
            "CUSTOMER_NOT_FOUND", ex.getMessage(), LocalDateTime.now());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

   @ExceptionHandler(MethodArgumentNotValidException.class)
   public ResponseEntity<ErrorResponse> handleValidation(
        MethodArgumentNotValidException ex) {
        ErrorResponse error = ErrorResponse.fromValidationErrors(ex);
        return ResponseEntity.badRequest().body(error);
    }
   // More @ExceptionHandler methods...
}
```

EXAMPLE: ERROR RESPONSE (JSON)

```
{
  "timestamp": "2025-05-15T10:30:00Z",
  "status": 404,
  "error": "Not Found",
  "code": "CUSTOMER_NOT_FOUND",
  "message": "Customer with id 123 not found",
  "path": "/api/customers/123",
  "details": null
}
```

KEY BENEFITS

- **Centralized exception handling** — Handle exceptions from all controllers in one place.
- **Consistent error responses** — Uniform error structure across the API improves client experience.
- **Cleaner controllers** — Controllers focus on business logic, not error handling.
- **Easier maintenance** — Update error handling logic in one place, applies everywhere.
- **Better reliability & security** — Avoid leaking internal details and return safe, meaningful messages.

COMMON EXCEPTIONS TO HANDLE

- **MethodArgumentNotValidException** — Validation errors on @RequestBody or @ModelAttribute
- **ConstraintViolationException** — Validation errors on @PathVariable / @RequestParam
- **ResourceNotFoundException** — Entity not found scenarios
- **AccessDeniedException** — Authorization / permission issues
- **Exception (Generic)** — Fallback for unexpected errors

TIP

Use **@RestControllerAdvice** for REST APIs so that @ResponseBody is applied automatically and responses are returned as JSON.

@EXCEPTIONHANDLER ANNOTATION

@ExceptionHandler is used inside a @ControllerAdvice class to handle specific exceptions and return a custom response to the client.

HOW IT WORKS

1. **Exception**
2. **Advice Catches**
3. **Handler Executes**
4. **Response Returned**

Exception	Handled By
IllegalArgumentException	handleIllegalArgument() -> 400
ResourceNotFoundException	handleNotFound() -> 404
AccessDeniedException	handleAccessDenied() -> 403
Exception (generic)	handleGeneralException() -> 500

EXAMPLE: @EXCEPTIONHANDLER IN ACTION

```
@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(ResourceNotFoundException.class)
  public ResponseEntity<ErrorResponse> handleNotFound(
      ResourceNotFoundException ex) {
    ErrorResponse response = new ErrorResponse(
        "RESOURCE_NOT_FOUND", ex.getMessage());
    return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
  }
  // multiple @ExceptionHandler methods can co-exist,
  // each targeting a different exception type
}
```

KEY TAKEAWAY @ExceptionHandler helps you handle specific exceptions in a global way, ensuring consistent, meaningful, and user-friendly error responses.

@EXCEPTIONHANDLER ANNOTATION

Handle Specific Exceptions with Dedicated Methods

@ExceptionHandler is used inside a @ControllerAdvice class to map a specific exception (or a list of exceptions) to a handler method.

PURPOSE

@ExceptionHandler lets you define methods that handle specific exceptions and return a custom response. It keeps error handling consistent and reusable.

HOW @EXCEPTIONHANDLER WORKS

1. **Exception Occurs** — An exception is thrown from controller, service or repository.
2. **Propagates Up** — The exception propagates up the call stack.
3. **ControllerAdvice Intercepts** — @ControllerAdvice class intercepts the exception.
4. **@ExceptionHandler Matches** — Spring matches the exception with the most specific @ExceptionHandler method.
5. **Handler Method Executes** — The matched handler method executes and builds the error response.
6. **Response Returned** — A structured response with the appropriate HTTP status is returned to the client.

EXAMPLE: @CONTROLLERADVICE WITH @EXCEPTIONHANDLER

```
@RestControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<ErrorResponse> handleValidationErrors(
        MethodArgumentNotValidException ex) {
    List<FieldError> fieldErrors = ex.getBindingResult().getFieldErrors();
    List<FieldErrorDetail> details = fieldErrors.stream()
        .map(err -> new FieldErrorDetail(err.getField(), err.getDefaultMessage()))
        .toList();
    ErrorResponse response = new ErrorResponse(
        "VALIDATION_FAILED", "Validation failed", details, LocalDateTime.now());
    return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
  }
  @ExceptionHandler(ResourceNotFoundException.class)
  public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
    ErrorResponse response = new ErrorResponse(
        "NOT_FOUND", ex.getMessage(), null, LocalDateTime.now());
    return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
  }
  @ExceptionHandler(Exception.class)
  public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
    ErrorResponse response = new ErrorResponse(
        "INTERNAL_SERVER_ERROR", "An unexpected error occurred", null, LocalDateTime.now());
    return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR);
  }
}
```

Key Points

- Methods are matched based on exception type.
- Most specific exception handler is chosen.
- Can handle multiple exceptions in one method.
- Return ResponseEntity to set status and body.
- Keep handlers focused, simple and consistent.

EXCEPTION TO HANDLER MAPPING

Exception Type	@ExceptionHandler Method
MethodArgumentNotValidException (Validation errors)	handleValidationErrors()
ResourceNotFoundException (Entity not found)	handleNotFound()
ConstraintViolationException (Bean validation)	handleConstraintViolation()
AccessDeniedException (Authorization failure)	handleAccessDenied()
Exception (or Throwable) (All other exceptions)	handleGeneric()

BENEFITS

- **Centralized error handling** — One place to handle exceptions across the application.
- **Consistent API responses** — Standard format improves client experience.
- **Cleaner controllers** — Controllers focus on business logic, not error handling.
- **Easy to maintain** — Update error handling logic in one place.
- **Better observability** — Easier logging, monitoring and alerting.

SUPPORTS MULTIPLE EXCEPTIONS

Handle multiple exceptions in a single method

```
@ExceptionHandler({
    IllegalArgumentException.class,
    NullPointerException.class })
public ResponseEntity<ErrorResponse> handleBadRequest(RuntimeException ex) {
    ErrorResponse response = new ErrorResponse(
        "BAD_REQUEST", ex.getMessage(), null, LocalDateTime.now());
    return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
}
```

Note

The method can handle one exception type or multiple exceptions.

EXAMPLE: ERROR RESPONSE (JSON)

```
{
  "timestamp": "2025-05-15T10:30:00Z",        ← Error time
  "status": 400,                              ← HTTP status code
  "error": "Bad Request",                     ← Short description
  "code": "VALIDATION_FAILED",                ← Application error code
  "message": "Validation failed",             ← Human readable message
  "path": "/api/customers",                   ← Request path
  "details": [
    { "field": "name", "message": "Name must not be blank" },
    { "field": "email", "message": "Email must be valid" }
  ]
}
```

BEST PRACTICES

- Provide meaningful error messages.
- Use appropriate HTTP status codes.
- Do not expose internal details to clients.
- Log exceptions with context.
- Return a consistent error response model.
- Test each exception scenario.

TRY IT YOURSELF — EXERCISE 2: HANDLER TODOS (STARTER)

Pre-lab Exercise 2 (fill-in-the-blank starter) -- complete a GlobalExceptionHandler sketch for validation and not-found.

STEP	WHAT TO DO
Goal	Complete a fill-in-the-blank GlobalExceptionHandler sketch that builds an ErrorResponse for validation and not-found failures.
Fill the Blanks	In notes/GlobalExceptionHandlerSketch.java, replace every _____ using the hints below.
Hints	@RestControllerAdvice; HttpStatus.BAD_REQUEST; correlation "lab-request-001"; HttpStatus.NOT_FOUND.
Controller Reminder	Write a note: controllers still need @Valid on @RequestBody -- without it, Bean Validation never runs.
Reflect	Confirm the happy-path GETs for CUS-1001 and CUS-1002 must still return 200 after these handlers exist.
Deliverable	GlobalExceptionHandlerSketch.java with every blank filled and the @Valid reminder written in notes.

notes/GlobalExceptionHandlerSketch.java -- STARTER (fill in every _____)

```
@_____
public class GlobalExceptionHandler {
  @ExceptionHandler(MethodArgumentNotValidException.class) @ResponseStatus(HttpStatus._____)
  ErrorResponse validation(MethodArgumentNotValidException ex) {
    return ErrorResponse.validation(_____, ex);  // TODO: include field violations
  }
  @ExceptionHandler(CustomerNotFoundException.class) @ResponseStatus(HttpStatus._____)
  ErrorResponse notFound(CustomerNotFoundException ex) {
    return ErrorResponse.notFound(ex.getCustomerId());
  }
}
```

TRY IT NOW Complete Exercise 2 before continuing -- see exercises/exercise-02-handler-todos.md.

STANDARD ERROR RESPONSE MODEL

A standard error response model ensures consistency across all APIs, making it easier for clients to understand, handle, and display errors.

Field	Description
timestamp	When the error occurred (ISO 8601 format).
status	HTTP status code.
error	Short reason phrase (e.g. Bad Request).
code	Application-specific error code.
message	Human-readable message for the client.
path	Request path where the error occurred.
correlationId / traceId	Unique request identifier for tracking.
errors / details	Optional field-level violation details.

Example Error Response (JSON)

```
{
  "timestamp": "2026-07-14T12:00:00Z",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_FAILED",
  "message": "Validation failed",
  "path": "/api/customers",
  "correlationId": "lab-request-001",
  "errors": [
    { "field": "email", "message": "must be valid" },
    { "field": "age", "message": "must be positive" }
  ]
}
```

KEY TAKEAWAY A standard error response model brings consistency, clarity, and reliability to your APIs and improves the overall integration experience.

Error Response Model

A consistent and standardized error response for REST APIs

Provides clear, actionable and machine-readable error information to API consumers.

Purpose

Ensure all APIs return consistent error responses that are easy to understand, debug and act upon.

STANDARD ERROR RESPONSE (JSON)

```
{
  "timestamp": "2025-05-15T10:30:00Z",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_FAILED",
  "message": "Validation failed for one or more fields.",
  "path": "/api/customers",
  "requestId": "8f3c2d9e-1a4b-4c15-8a3d-9b7e1c2f0a11",
  "details": [
    {
      "field": "name",
      "rejectedValue": "",
      "message": "Name must not be blank"
    },
    {
      "field": "email",
      "rejectedValue": "invalid-email",
      "message": "Email must be a valid email address"
    }
  ],
  "errors": []
}
```

FIELD DESCRIPTION

timestamp	Date and time when the error occurred (ISO 8601 format)
status	HTTP status code of the error
error	Short description of the HTTP status
code	Application-specific error code (unique and consistent)
message	Human-readable message that describes the error
path	API endpoint where the error occurred
requestId	Unique ID for the request (useful for tracking and support)
details	Array of field or object level validation errors (if applicable)
errors	Array of global or business rule errors (if applicable)

details

field	Name of the field that failed validation
rejectedValue	Value that was provided by the client
message	Validation message for the field

HTTP STATUS CODE MAPPING

Status Code	When to Use	Example Error Codes
400 Bad Request	Validation errors, invalid request data	VALIDATION_FAILED, INVALID_INPUT
401 Unauthorized	Authentication required or failed	AUTHENTICATION_FAILED, TOKEN_MISSING
403 Forbidden	Authenticated but not authorized	ACCESS_DENIED, INSUFFICIENT_PRIVILEGES
404 Not Found	Resource not found	RESOURCE_NOT_FOUND
409 Conflict	Resource conflict, duplicate data	CONFLICT, DUPLICATE_RESOURCE
422 Unprocessable Entity	Semantically incorrect data	BUSINESS_RULE_VIOLATION, UNPROCESSABLE_ENTITY
500 Internal Server Error	Unexpected server error	INTERNAL_SERVER_ERROR

EXAMPLE SCENARIOS

Validation Error (400) Request data is invalid or missing required fields	code: VALIDATION_FAILED status: 400
Authentication Error (401) User is not authenticated or token is invalid	code: AUTHENTICATION_FAILED status: 401
Authorization Error (403) User does not have permission to access the resource	code: ACCESS_DENIED status: 403
Resource Not Found (404) Requested resource could not be found	code: RESOURCE_NOT_FOUND status: 404
Server Error (500) Unexpected error on the server	code: INTERNAL_SERVER_ERROR status: 500

GLOBAL / BUSINESS RULE ERRORS (errors [])

```
"errors": [
  {
    "code": "BUSINESS_RULE_VIOLATION",
    "message": "Customer cannot be deleted as it has
               active orders."
  }
]
```

KEY BENEFITS

- **Consistency** — Same structure across all APIs
- **Developer Friendly** — Clear, structured and documented
- **Better Client Experience** — Easy to parse and handle
- **Easier Maintenance** — Centralized error handling
- **Improved Observability** — Use requestId for tracking
- **Security** — No internal details exposed

BEST PRACTICES

- Always return a consistent error response structure.
- Use meaningful and unique application error codes.
- Do not expose sensitive information or stack traces.
- Include a requestId for correlation and debugging.
- Provide clear messages that help clients fix the issue.
- Use appropriate HTTP status codes.
- Document all error codes and messages.
- Keep the response concise and to the point.

A well-designed error response model improves API reliability, debuggability and overall user trust.

TRY IT YOURSELF — EXERCISE 3: ERROR RESPONSE ENVELOPE

Pre-lab Exercise 3 -- sketch the exact ErrorResponse JSON envelope for a 400 validation failure and a 404 not-found.

STEP	WHAT TO DO
Goal	Sketch the exact ErrorResponse JSON envelope for two real failure scenarios before any handler code exists.
Sketch #1 (400)	In notes/error-envelope.md, write a full validation-failure envelope, including violations and correlationId: "lab-request-001".
Reference Check	Confirm every required field appears: timestamp, status, error, message, path, correlationId, violations.
Sketch #2 (404)	Sketch the envelope returned for GET /api/customers/CUS-9999 when the customer does not exist.
Safety Check	Re-read both sketches -- confirm message never contains a stack trace, SQL fragment, or internal class/table name.
Deliverable	notes/error-envelope.md with a safety-checked 400 sketch and a 404 sketch.

Target shape -- 400 validation example

```
{
  "timestamp": "2026-07-27T09:15:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed",
  "path": "/api/customers",
  "correlationId": "lab-request-001",
  "violations": [ { "field": "email", "message": "must be a valid email" } ]
}
```

TRY IT NOW Complete Exercise 3 before continuing -- see exercises/exercise-03-error-envelope.md.

ERROR CODES AND MESSAGES

Well-defined error codes and clear messages help clients understand what went wrong and how to fix the issue quickly.

Code	HTTP Status	Message	Description
VALIDATION_FAILED	400	Validation failed for one or more fields.	Request parameters or payload validation failed.
RESOURCE_NOT_FOUND	404	The requested resource was not found.	Resource does not exist or is unavailable.
ACCESS_DENIED / UNAUTHORIZED	403 / 401	You do not have permission / authentication required.	Authenticated but not authorized, or missing auth.
DUPLICATE_RESOURCE	409	Resource already exists.	Attempt to create a duplicate resource.
INTERNAL_SERVER_ERROR	500	An unexpected error occurred. Please try again later.	Server-side unexpected condition.

MESSAGE DESIGN TIPS

Use simple language **Explain what & how to fix** **Include field-level details** **Avoid internal jargon**

KEY TAKEAWAY Consistent error codes and user-friendly messages make your APIs easier to integrate, faster to troubleshoot, and better for everyone.

TRY IT YOURSELF — EXERCISE 4: EXCEPTION TO STATUS MAP

Pre-lab Exercise 4 -- map five real Lab 29 failure cases to the HTTP status code each should return.

STEP	WHAT TO DO
Goal	Document how each of five real Lab 29 failure cases maps to an HTTP status code.
Fill the Map	In notes/exception-status-map.md, map: Bean Validation failure, customer not found, duplicate create, illegal status transition, and unhandled exception.
Reference Check	Compare against: 400, 404, 409, 409-or-422, and 500 (safe fallback).
Justify	Pick 409 or 422 for illegal status transition and justify the choice in one sentence.
Handler Type	Note that all five cases are handled by one @RestControllerAdvice / GlobalExceptionHandler class, not five separate try/catch blocks.
Deliverable	notes/exception-status-map.md with all five cases mapped, the handler type named, and the transition justification written.

Reference -- exception-status-map.md

```
Bean Validation failure          -> 400 Bad Request
Customer not found (CUS-9999)    -> 404 Not Found
Duplicate create (CUS-1001)      -> 409 Conflict
Illegal status transition        -> 409 or 422 (justify your pick)
Unhandled exception              -> 500 Internal Server Error (safe fallback)
```

TRY IT NOW Complete Exercise 4 before continuing -- see exercises/exercise-04-exception-status-map.md.

API ERROR STANDARDS

Consistent error standards make APIs easier to implement, integrate, and support -- improving developer experience and system reliability.

STANDARD PRINCIPLES

Consistent — use the same error structure and fields across all APIs.

Clear & Actionable — provide clear messages that help clients understand and fix issues.

Secure — do not expose internal details, stack traces, or sensitive data.

Appropriate — use correct HTTP status codes for each error scenario.

Traceable — include a correlation/trace ID to help track and troubleshoot issues.

HTTP STATUS CODE STANDARDS

Status	Meaning
400 Bad Request	Invalid request syntax or validation failure.
401 Unauthorized	Authentication required and has failed.
403 Forbidden	Authenticated but not authorized to access.
404 Not Found	Requested resource does not exist.
409 Conflict	Conflicts with current state (e.g. duplicate).
422 Unprocessable Entity	Semantic errors or business rule violations.
500 Internal Server Error	Unexpected server-side error.

ERROR CODE GUIDELINES

UPPER_SNAKE_CASE codes

Unique, stable, documented

Grouped by domain (VALIDATION_*, AUTH_*)

Never change published codes -- version instead

KEY TAKEAWAY Define and follow a consistent error standard across all APIs to ensure clarity, reliability, and better client integration.

API Error Standards

Consistent, Predictable, and Machine-Readable Errors

Following standard error practices helps developers quickly understand, debug, and handle errors while providing a better experience for API consumers.

Why Standardize Errors?

- Improves developer experience
- Simplifies client-side error handling
- Enables automation and monitoring
- Promotes consistency across services
- Enhances security (no information leakage)

Key Principles

- Use consistent structure and fields
- Use appropriate HTTP status codes
- Provide clear, actionable messages
- Do not expose internal details
- Support localization and extensibility

Standard Error Response Model

```
{
   "timestamp": "2025-05-20T14:35:22Z",
   "status": 400,
   "error": "Bad Request",
   "code": "VALIDATION_FAILED",
   "message": "Validation failed for one or more fields.",
   "path": "/api/v1/customers",
   "traceId": "550e8400-e29b-41d4-a716-446655440000",
   "errors": [
      {
         "field": "email",
         "rejectedValue": "johnexample.com",
         "message": "Invalid email format"
      }
   ]
}
```

Field	Description
timestamp	When the error occurred (ISO 8601)
status	HTTP status code (integer)
error	Short description of the HTTP status
code	Application-specific error code (stable)
message	Human-readable message (developer/user friendly)
path	Request path when the error occurred
traceId	Unique request identifier for tracing
errors	(Optional) Field-level or detailed error information

Use Appropriate HTTP Status Codes

Status Code	Meaning	When to Use
200 OK	Success	Request succeeded
201 Created	Resource created	Resource created successfully
400 Bad Request	Client error	Invalid request syntax or validation failed
401 Unauthorized	Authentication required	Missing or invalid authentication
403 Forbidden	Access denied	Authenticated but not authorized
404 Not Found	Resource not found	Requested resource does not exist
409 Conflict	Conflict with current state	Duplicate resource or state conflict
422 Unprocessable Entity	Semantic validation error	Request is well-formed but invalid
500 Internal Server Error	Server error	Unexpected server-side error
503 Service Unavalable	Temporary overloading	Service temporarily unavailable

Error Code Design Guidelines

Stable & Versioned
Define a clear set of error codes and keep them backward compatible.

Namespaced
Use a consistent naming convention (e.g., USER_NOT_FOUND).

Meaningful
Codes should be descriptive and actionable for clients.

Documented
Maintain a public error code catalog with meanings and resolutions.

Common Error Code Categories (Examples)

Category	Example Error Codes
Validation	VALIDATION_FAILED, INVALID_FIELD, MISSING_FIELD
Authentication	UNAUTHORIZED, INVALID_CREDENTIALS, TOKEN_EXPIRED
Authorization	FORBIDDEN, INSUFFICIENT_PERMISSIONS
Resource	NOT_FOUND, ALREADY_EXISTS, RESOURCE_CONFLICT
Business Rule	BUSINESS_RULE_VIOLATION, INVALID_STATE, LIMIT_EXCEEDED
Rate Limiting	TOO_MANY_REQUESTS, RATE_LIMIT_EXCEEDED
Server	INTERNAL_ERROR, SERVICE_UNAVAILABLE, TIMEOUT_ERROR

Best Practices

- Return machine-readable errors (JSON).
- Keep messages clear, concise, and actionable.
- Never expose stack traces or internal details.
- Include a traceId for correlation and support.
- Log full details on the server, return safe details to clients.
- Use consistent structure across all APIs and services.
- Version your API and error schema when needed.

Localization Support (Optional)

Provide error messages in the client's preferred language.

```
"message": "Invalid email format",
"message_i18n": {
   "en": "Invalid email format",
   "fr": "Format d'e-mail invalide",
   "es": "Formato de correo inválido"
}
```

The Goal

Deliver consistent, secure, and meaningful error responses that help both humans and machines handle failures gracefully.

Do

- Be consistent
- Be helpful
- Be secure

Don't

- Return HTML error pages
- Expose internals or stack traces
- Use vague error messages

Remember

Good error standards = Better integrations, faster debugging, and happier developers!

LOGGING VALIDATION FAILURES

Logging validation failures helps teams quickly identify bad requests, analyze root causes, and improve data quality and API reliability.

WHAT TO LOG

- Timestamp, request/correlation ID, and client IP.
- HTTP method, request path, and status code returned.
- Validation error details: field, rejected value, message.
- User or client identifier, if available.

Never log sensitive data like passwords, tokens, or PII.

Logging inside the handler

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidation(
    MethodArgumentNotValidException ex,
    @RequestHeader(value = "X-Correlation-Id",
        required = false) String requestId) {
  var errors = ex.getBindingResult().getFieldErrors();
  log.warn("VALIDATION_FAILURE | requestId={} errors={}",
      requestId, errors);
  return ResponseEntity.badRequest().body(
      ErrorResponse.of("VALIDATION_FAILED", errors));
}
```

BEST PRACTICES

- **Log at WARN level**
- **Use structured (JSON) logs**
- **Include a correlation ID**
- **Never log full request bodies**

KEY TAKEAWAY Logging validation failures with proper context and structure helps you monitor, secure, and improve your APIs for a better client experience.

VALIDATION AND SECURITY

Strong validation and security controls protect your APIs from invalid, malicious, and unauthorized requests while delivering clear feedback.

VALIDATION LAYERS (DEFENSE IN DEPTH)

1. **Client-Side**
   Instant UI feedback
2. **API Request**
   Bean Validation
3. **Service / Business**
   Business rules, state
4. **Database**
   Constraints, final safeguard

SECURITY CONTROLS

Authentication & Authorization — verify identity (JWT/OAuth2) and permission before processing.

Input Validation & Sanitization — validate all inputs; sanitize to prevent SQL injection and XSS.

Rate Limiting & HTTPS — limit abusive requests; encrypt data in transit.

WHAT NOT TO DO

- Do not trust client-side validation only.
- Do not expose stack traces or internal system details.
- Do not log sensitive data (passwords, tokens, PII).
- Do not accept unexpected data silently.

KEY TAKEAWAY Robust validation and security protect your APIs, your data, and your users. Validate early, secure by design, and respond with clear, safe, consistent errors.

Validation and Security in Spring Boot APIs

Validate early. Authenticate securely. Authorize wisely.

A strong combination of input validation and security protects your applications from bad data, malicious attacks, and unauthorized access.

Goal
Ensure only valid, safe, and authorized requests are processed by the API.

spring boot

REQUEST PROCESSING FLOW

1. CLIENT REQUEST
Client sends an HTTP request

2. SECURITY FILTER CHAIN
- Authentication
- Authorization
- CSRF Protection
- Rate Limiting
- Headers Security

3. INPUT VALIDATION
- Bean Validation (@Valid, @Validated)
- Custom Validators
- Parameter Validation
- Sanitization

4. CONTROLLER
If validated and authorized, request reaches controller

5. SERVICE & BUSINESS LOGIC
Business rules are applied

6. SUCCESS RESPONSE
Return success response

ON FAILURE
- Validation Error → 400 Bad Request
- Authentication Error → 401 Unauthorized
- Authorization Error → 403 Forbidden

Return standardized error response

INPUT VALIDATION

BEAN VALIDATION (JSR 380)

```
public class CreateUserRequest {
    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Invalid email")
    private String email;

    @Size(min = 8, message = "Password must
      be at least 8 characters")
    private String password;

    @Min(value = 18, message = "Must be at
     least 18 years old")
    private Integer age;
}
```

VALIDATION LAYERS
- Request Body Validation — @Valid on @RequestBody
- Path Variable Validation — @Pattern, @Min, @Max
- Query Parameter Validation — @RequestParam + Constraints
- Method Parameter Validation — @Validated on Controller/Service
- Custom Business Validation — Custom @Constraint

HANDLING VALIDATION ERRORS

```
{
  "timestamp": "2025-05-15T10:30:002",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_FAILED",
  "message": "Validation failed for one or more fields",
  "path": "/api/users",
  "errors": [
    {"field": "email", "rejectedValue": "abc", "message": "Invalid email"},
    {"field": "password", "rejectedValue": "123", "message": "Password must be
      at least 8 characters"}
  ]
}
```

COMMON VALIDATION ERRORS
- MethodArgumentNotValidException (Invalid @RequestBody)
- ConstraintViolationException (Invalid path/query params)
- TypeMismatchException (Invalid data type)
- Custom Business Rule Violations

SECURITY LAYERS

AUTHENTICATION
Verify the identity of the user (JWT, OAuth2, Basic Auth)

AUTHORIZATION
Ensure the user has permission (RBAC, @PreAuthorize)

INPUT SECURITY
Validate and sanitize all inputs to prevent injection and other attacks

TRANSPORT SECURITY
Use HTTPS/TLS to protect data in transit

SECURE HEADERS
Enable security headers (HSTS, X-Frame-Options, Content-Type-Options, etc.)

CSRF PROTECTION
Protect state-changing operations (for session-based auth)

RATE LIMITING
Prevent abuse and brute force attacks

SECURITY BEST PRACTICES

- Always validate all external inputs
- Never trust client data
- Use Bean Validation + Custom Validation
- Use parameterized queries / JPA to prevent SQL Injection
- Use HTTPS everywhere
- Store passwords using strong hashing (BCrypt, Argon2)
- Use least privilege access
- Keep dependencies and libraries updated
- Log security events (login failures, validation failures, access denied)
- Test security with automation (Pen testing, SAST/DAST)

COMMON THREATS MITIGATED

Injection Attacks
SQLi, XSS, Command Injection

Broken Authentication
Weak login, credential stuffing

Broken Authorization
IDOR, privilege escalation

Sensitive Data Exposure
Data leakage, weak encryption

Denial of Service
Rate limiting, resource abuse

CSRF Attacks
Forged requests

KEY TAKEAWAY

Strong validation ensures data is correct.
Strong security ensures your system is safe.
Together, they build robust and trustworthy APIs.

Validate inputs early to maintain data quality. Secure every request to protect your application and users.
Validation + Security = Reliable, Safe and Enterprise-Ready Applications.

ERROR HANDLING BEST PRACTICES

Good error handling improves reliability, security, and user experience. Follow these best practices to build robust and maintainable APIs.

#	Practice	What It Means
1	Validate Early	Validate all inputs as early as possible and fail fast with meaningful messages.
2	Use Appropriate HTTP Status Codes	Return the correct status that matches the error type (400 bad request, 404 not found, etc.).
3	Use a Standard Error Response Model	Maintain a consistent structure with fields like timestamp, status, code, message, path, traceId.
4	Do Not Expose Internal Details	Avoid exposing stack traces, SQL queries, or internal class names in responses.
5	Log with Context	Log errors with requestId/traceId, userId, path, and useful metadata for faster troubleshooting.
6	Handle Exceptions at the Right Level	Handle exceptions at a central layer (@ControllerAdvice); avoid try-catch in every method.

BAD VS GOOD ERROR MESSAGE

Bad	Good
"Error while processing request" -- status 500, no detail, not actionable.	"Email must be a valid email address" -- status 400, clear, specific, actionable.

KEY TAKEAWAY Following error handling best practices ensures your APIs are reliable, secure, and user-friendly, while making them easier to monitor, debug, and maintain.

TRY IT YOURSELF — EXERCISE 5: LAB 29 READINESS CHECK

Pre-lab Exercise 5 (capstone) -- confirm lab29-crm is ready to start as the Week 3 error-contract capstone.

STEP	WHAT TO DO
Goal	Confirm lab29-crm is ready to start as the Week 3 error-contract capstone.
Dependencies	In notes/lab29-readiness.md, list the Lab 14 / Lab 16 concepts and the Lab 25-28 Spring Boot surface this lab builds on.
Project Path	Write down the real project path: examples/lab29-crm.
Evidence Plan	Plan curl evidence for 400, 404, and 409 responses, plus the happy paths for CUS-1001 and CUS-1002, all correlated via lab-request-001.
Week Boundary	Explicitly note that Week 4 topics (Kafka, React, PostgreSQL) are out of scope for this pre-lab and for Lab 29's timed path.
Deliverable	notes/lab29-readiness.md with dependencies listed, the project path written, and Week 4 explicitly excluded.

Evidence to capture before Lab 29

```
$ curl -i -X POST http://localhost:8080/api/customers -d '{...}'   // expect 400 + violations[]
$ curl -i http://localhost:8080/api/customers/CUS-9999             // expect 404 RESOURCE_NOT_FOUND
$ curl -i -X POST http://localhost:8080/api/customers -d '{...CUS-1001...}'  // expect 409 duplicate
$ curl -i http://localhost:8080/api/customers/CUS-1001             // expect 200, Amina Khan, ACTIVE
# Every response correlated via X-Correlation-Id: lab-request-001
```

TRY IT NOW Complete Exercise 5 before continuing -- see exercises/exercise-05-lab29-readiness.md.

LAB OVERVIEW -- VALIDATION AND EXCEPTION HANDLING

Lab 29: Validation and Exception Handling -- Northstar CRM Error Contracts

BUSINESS SCENARIO

- Agents and integrations will send bad JSON. Leadership freezes the rule: no API error path may return framework-default HTML stack traces or ad-hoc Map bodies to React.
- You own the contract for Amina (ACTIVE), Ravi (PROSPECT), unknown IDs, duplicates, and illegal lifecycle transitions.
- This unifies Lab 14's DTO/validation ideas and Lab 16's exception-handler ideas into one stable Spring Boot contract every client relies on.

LEARNING OBJECTIVES

- Annotate CRM request DTOs with Bean Validation constraints.
- Trigger validation with @Valid / @Validated on controller methods.
- Map field and object-level violations into a stable ErrorResponse.
- Centralize exception handling with @RestControllerAdvice / @ExceptionHandler.
- Align HTTP status codes with business exceptions (404, 409, 400/422).
- Preserve correlation IDs such as lab-request-001 in error responses.

WHAT YOU BUILD

- Copy lab28-crm to lab29-crm; add spring-boot-starter-validation; define ErrorResponse (+ FieldViolation); annotate CustomerRequest/StatusUpdateRequest; enable @Valid on controllers; implement GlobalExceptionHandler for validation, not-found, duplicate, illegal transition, and a safe 500 fallback; write MockMvc tests asserting status and body shape.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; CUS-9999 proves not-found handling; correlation ID lab-request-001 appears on every error envelope.

LAB TASKS AND DELIVERABLES

Northstar CRM Error Contracts -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch lab28-crm to lab29-crm; add spring-boot-starter-validation; define ErrorResponse + FieldViolation.
2	Annotate CustomerRequest (@NotBlank, @Pattern, @Email) and StatusUpdateRequest (@NotNull).
3	Enable @Valid on create and status-update controller methods.
4	Prove validation failures with curl -- observe framework-default, then custom envelope.
5	Implement GlobalExceptionHandler for MethodArgumentNotValidException -> 400.
6	Map domain exceptions: not-found -> 404, duplicate -> 409, illegal transition -> 400/422.
7	Add a safe generic Exception -> 500 fallback; log full detail server-side only.
8	Write MockMvc tests (status + jsonPath body); document a Lab 14/16 unify note.

EXPECTED DELIVERABLES

- lab29-crm with Bean Validation + GlobalExceptionHandler + ErrorResponse.
- Automated tests for validation and not-found envelopes.
- Successful-path evidence (CUS-1001, CUS-1002).
- Controlled-failure evidence (400/404/409 + envelope).
- Lab 14/16 unify note; no secrets or target/ committed.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY Happy-path tests without envelope assertions are incomplete; leaking a stack trace to a client loses security/production marks even if the happy path works.

TRY IT YOURSELF — EXERCISE 6: MOCKMVC BODY ASSERTIONS

Pre-lab Exercise 6 -- plan MockMvc tests that assert JSON response fields, not just the HTTP status code.

STEP	WHAT TO DO
Goal	Plan MockMvc tests that assert JSON response fields, not just the HTTP status code.
Four Cases	In notes/mockmvc-body-plan.md list: invalid POST, GET CUS-9999, duplicate-create CUS-1001, and happy GET for Amina (CUS-1001).
Failure Assertions	For each failure case, name the assertions: status, message/error, and correlationId all exist in the body.
Security Coexistence	If Lab 28 security is complete, note that tests may need auth headers -- do not remove security to make a test pass.
Boundary	Do not implement full MockMvc test classes yet -- this is a planning exercise; Lab 29 builds the real tests.
Deliverable	notes/mockmvc-body-plan.md with all four cases and their envelope-field assertions listed.

Target assertion style (MockMvc + jsonPath)

```
mockMvc.perform(post("/api/customers")
        .contentType(APPLICATION_JSON).content(invalidCustomerJson))
    .andExpect(status().isBadRequest())
    .andExpect(jsonPath("$.status").value(400))
    .andExpect(jsonPath("$.correlationId").exists())
    .andExpect(jsonPath("$.violations").isArray());
```

TRY IT NOW Complete Exercise 6 before continuing -- see exercises/exercise-06-mockmvc-body-assertions.md.

MODULE 29 SUMMARY

In this module, we learned how to validate inputs, handle exceptions globally, and return standardized error responses to build secure, reliable, user-friendly APIs.

KEY CONCEPTS COVERED

Validate Early

Validate all inputs using Bean Validation (JSR 380) to catch invalid data early.

Handle Globally

Use @ControllerAdvice and @ExceptionHandler to handle exceptions across the API.

Consistent Error Model

Return a standard error response with timestamp, status, code, message, path, traceId.

Secure by Design

Never expose internal details or stack traces; log sensitive information securely.

Monitor & Improve

Log validation failures and exceptions with context to monitor and improve APIs.

Better User Experience

Provide clear, actionable error messages that help clients fix issues quickly.

MODULE FLOW RECAP

1. **Bean Validation**
2. **@Valid on Controllers**
3. **@ControllerAdvice**
4. **Standard ErrorResponse**
5. **Lab: Northstar CRM**

KEY TAKEAWAY Effective validation and global exception handling are essential for building enterprise-grade APIs. They ensure data integrity, system security, and a great developer and user experience.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of validation annotations, exception handling, standard error responses, and HTTP status codes.

1. Which annotation is used to trigger input validation in Spring Boot?

A. @Valid

B. @ValidateInput

C. @ValidRequest

D. @CheckInput

2. Where should you handle exceptions centrally in a Spring Boot REST API?

A. In each service method

B. In each controller method

C. Using @ControllerAdvice and @ExceptionHandler

D. In the repository layer

3. What is the purpose of a standard error response model?

A. To hide exception details from logs

B. To provide inconsistent responses

C. To return consistent, structured error information to clients

D. To replace HTTP status codes

4. Which HTTP status code is most appropriate for validation failures?

A. 200 OK

B. 400 Bad Request

C. 404 Not Found

D. 500 Internal Server Error

KEY TAKEAWAY @Valid triggers validation; @ControllerAdvice/@ExceptionHandler centralize handling; a standard error model returns consistent, structured information with 400 for validation failures.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on error-handling best practices, testing tools, Bean Validation annotations, and internal-detail exposure.

5. Which of the following is a best practice for error handling?

A. Expose stack traces in responses

B. Return generic error messages

C. Log errors with context (requestId, path, userId, etc.)

D. Use different error formats in different APIs

6. Which tool is commonly used for API testing in this module?

A. Swagger UI

B. Postman

C. Jenkins

D. Elasticsearch

7. Which of the following is NOT a validation annotation in Bean Validation (JSR 380)?

A. @NotNull

B. @Email

C. @Size

D. @NotEmptyString

8. Why should internal error details not be exposed to API clients?

A. It improves response time

B. It reduces log size

C. It protects security and prevents information leakage

D. It is required by the HTTP specification

KEY TAKEAWAY Log with context, test with Postman, know the real Bean Validation annotations (@NotEmptyString is not one), and hide internal details to protect security.

"Validate early. Handle gracefully. Respond consistently."

MODULE 29 COMPLETE

Validation and Global Exception Handling

You can now validate request data with Bean Validation, centralize exception handling with @ControllerAdvice, and return standardized, secure, user-friendly error responses.